

Application of Deep Learning Models for Stock Trend Projection:
Using Sentiment Analysis and Machine Learning to Understand Stock Validity

Ayomiro Aderin, Cameron Graves, Abdul Seidu

CSC 499

15 April 2024

Approved by:

Musbah Abdulgader
[Research Advisor Name]

Contents

ABSTRACT	4
INTRODUCTION	5
<i>Goal</i>	6
<i>Objectives</i>	6
<i>Research Questions</i>	7
LITERATURE REVIEW	7
METHODOLOGY	8
EXPECTED RESULTS	9
DESIGN DETAILS	9
TEST PLANS AND RESULTS	11
TIMELINE	11
INDIVIDUAL PARTS AND LABOR	12
EXPERIMENT	12
RESULTS	13
RECOMMENDATIONS	14
CONCLUSION	15
WORKS CITED	16
APPENDIX A – ARTIFICIAL NEURAL NETWORK	17

APPENDIX B – SENTIMENT ANALYSIS	23
APPENDIX C – MAIN PROGRAM	28

Abstract

This research project aims to expand the knowledge base of deep learning models by simplifying the complexities of the Stock Exchange for first-time investors. While the basics of stock trading are understood, the factors driving stock value changes remain enigmatic to many. Leveraging the rise of deep learning, this project seeks to detect patterns in financial news and provide data-driven guidance to empower investors. It aspires to achieve a 75% prediction accuracy rate by identifying the relationship between financial news and stock values, deploying sentiment analysis, and combining multiple deep learning models. The research builds on previous work highlighting the effectiveness of deep learning models and neural networks in stock prediction and explores the potential for further enhancing predictive accuracy. The methodology involves parsing quantitative and textual data, and the project's expected outcome is a user-friendly website offering comprehensive stock analyses to facilitate informed investment decisions. This project signifies a step toward bridging the gap between human understanding and machine learning capabilities, ultimately advancing predictive analysis in the stock market context.

Introduction

The Stock Exchange appears to the common eye as an enigmatic indicator of a company's success. While the intricacies are unknown, the basic principle of buying and selling stocks in response to the rise or descent of a stock's value is understood. That being said, the intricacies of what contributes to a property's appreciation or depreciation are not fully understood by the general populous. Too often, amateur investors don't grasp the nuances that would cause a company's value to change and as such, respond in ignorance to its changing tide. These hasty actions cause these investors to either miss out on big opportunities to gain money or lose their investment to a steep decline in value. As such, those who are interested in engaging with the Stock Exchange are wholly disappointed and discouraged from participating, opting to save their money on more immediate needs.

With the rise of deep learning algorithms, it's becoming easier for programs to determine human pattern and operate based upon this information. Within the world of finance, the movement of the stock exchange is wholly dependent upon the happenings that surround the company it evaluates. This project recognizes that there's a correlation between financial news and the number of shares sold on a traditional stock day. Therefore, this project aims to remedy the relationship between investors and the stock market by detecting such patterns and advising users upon which course of action they should take based upon the information fed into the software. In doing so, it aspires to empower investors with the knowledge and insights needed to navigate the stock market more effectively.

The motivation of this project stems from the rise in the integration of machine learning. Every day, businesses are utilizing deep learning algorithms to better interact with the vast amounts of data located on the Internet. Social media sites such as YouTube and TikTok take a

user's watch history to determine what other videos they would be interested in. Google takes your search data to provide information that is pertinent to one's curiosity. Online stores take past browsing experiences into mind when curating your current experience. Predictive analysis has traditionally been applied in more low stakes arenas, serving the purpose of catering to each individual Internet experience. By applying this science to the world of stock trading, it may expose the expanded possibilities of machine learning through its use in teaching first-time investors how to determine the direction of a company's stock.

Goal

The goal of this research project is to illustrate a solid correlation between financial news content and the performance of company stock values. By utilizing deep learning models and data analysis techniques, the project aims to develop a predictive model that can forecast future stock movements with a high level of accuracy, targeting a goal of achieving with a 75% prediction accuracy rate.

Objectives

- To identify the relationship between financial news content and stock exchange values
- To achieve a minimum accuracy rate of 75% in determining the direction of a company's stock.
- To concisely describe the factors that determine a stock's direction to a general audience
- To employ sentiment analysis to discover terms and phrases that are associated with the reputation of a company
- To leverage deep learning models to determine the direction of a company's stock
- To combine two deep learning algorithms to create a hybrid model that associates the reputation of a company with its stock direction

Research Questions

Throughout this project, the research team aims to answer the following questions throughout its exploration of deep learning models and their relationship with predictive analytics: How can machine learning associate qualitative data with quantitative data to form conjectures? What are the strongest determining factors that deep learning models can perceive to help form its strongest predictions? Can the integration of sentiment analysis and deep learning models provide a more accurate and reliable method for predicting stock market trends in comparison to traditional quantitative models?

Literature Review

Ever since machine learning principles started their integration into society, academics have been exploring how deep learning models could study stock market patterns to predict their outcomes. Gurjeet Singh conducted a study with various methods of machine learning to determine which model could best accurately describe the next few days on the stock exchange. Upon drawing his conclusions, Singh recognized that the two strongest deep learning models were the linear regression model and the artificial neural network models [1]. In contrast, Payal Soni and his cohort theorized that while singular models could only yield a 50% to 70% accuracy rate, “the fusion of two or more neural networks could be applied, to increase the predictive accuracy value of stock prices and stock market trends” [2]. Given this insight, it shows that machine learning is capable of discerning the direction of a stock value. However, it could stand to benefit from pairing with another machine learning model.

When looking through Dr. Somanchi Khrisna’s article, *The Use of Deep Learning for Predictive Analysis*, they approach the topic by surveying a host of quantitative deep learning models to examine stock data. However, given that they don’t incorporate any natural language processing models, this experiment limits itself in its scope, an attribute that Dr. Khhrisna

recognizes [3]. One report, conducted by Marxia Oli Sigo, focused upon predictive analytics within the scope of the Indian Stock Exchange. What this report recognized was that artificial neural network enhanced stock price movement predictability [4]. This contribution reinforces the idea that more complex recognitions of patterned data have resulted in machine learning algorithms to anticipate the next steps of the stock market. With this fertile ground to stand upon, this project aims to continue where these studies stopped by combining deep learning models and utilizing a combination of numerical and textual data to help these programs recognize correlation and causation.

Methodology

To determine our end result, we will employ deep learning models to parse through our data. Our line regression model will examine .csv files relating to stock market values such as the opening price, the closing price, the highest price, and the lowest price. This element will present the stock asset's trajectory with regards to whether it will continue to rise or fall. The sentiment analysis model will determine which words help indicate if a stock property will appreciate or depreciate in value. Quite often, financial news helps traders determine the reputation of certain companies. By analyzing the language these articles present, we should be able to figure out if the company is in its better days.

When gathering data, we aim to collect two forms of data and engage with them through methods related to supervised learning. To receive quantitative data, we aim to collect a host of .csv files from NASDAQ and Nifty 50. Conversely, we will create a web scraper using TensorFlow and utilize a financial news API to gather news articles, pick out sentimental

language, and store the words in .csv files. With these combined data sets, the goal is to utilize the information to determine the impending direction of a stock property.

Expected Results

The project's expected deliverable is a user-friendly website that will assist a client in determining the stability of a company's stock. When investigating an asset, the website will sift through up to five years' worth of financial data, stock reports, and company history to provide users with a comprehensive analysis of a property. When a user searches for a specific stock property, the website will succinctly present a determined course of action, backed by data-centric rationale. This feature is crucial for first-time investors with no knowledge of the components that contribute to a company's rise or fall on the stock exchange. By offering informed perspectives of these predictions, the website aspires to inspire first-time users to make informed financial decisions surrounding the stock market.

Design Details

During the design of this website, the development team made the assumption that there is a correlation between stocks news and stock data. This is based upon the idea that quite often, the news influences how traders make their financial decisions. In addition, through creating the hybrid model, it is assumed that there is a underlying pattern within this correlation that can be exploited for deterministic purposes. This notion comes from a basic understanding of how the stocks work. Given that the stock market has been around for centuries, there must be factors one can closely observe as a result of historical data. Finally, it is assumed that once the software makes a prediction, that prediction can help stock watchers invest their money more intelligently.

The fundamental function of the website is to analyze stock market information, ultimately figuring out where a company's stock is headed and providing users with information that would allow for them to make sound financial decisions. Upon entering the website, a user will have their individual log-in. When logged in, users can keep their eye on up to three stock market properties. The model will send a summarized update of each property and the user would be able to select any of the properties to get a deeper understanding behind the website's decision. The goal is ease-of-access. If a user wants the quick prediction, they can get it and leave the website. However, if they want a better understanding, they can leave the homepage to better understand the factors that went into the prediction. That way, users can slowly learn about what goes into making stock predictions.

The prototypes developed include small-scale deep learning models for comparative use. So far, the team has developed linear regression models (single and multiple), a support vector model, a random forest model, and an artificial neural network (ANN) model. To determine which model would better serve our needs and conjure the greatest accuracy rate, we've written code that analyzes breast cancer data to determine the strength of each model's predictability. In addition, we've developed a sentiment analysis model that reads headlines from a stock news website to determine its impact on the stock.

To develop and maintain this website, the cost analysis includes development tools, hardware, software subscriptions, and labor. Given that this is a student project, the team aims to locate cost-effective solutions when it comes to developing this website. That being said, aspects such as website hosting and certain APIs may require monetary investment. However, the software and their libraries (Visual Studio Code, TensorFlow, Anaconda...) are traditionally

open-source and available for all users. As such, any cost would traditionally come from the latter stages of website development.

Test Plans and Results

To determine where a company's stock property is headed, the team will enter the ticker name for a company. Upon entry, the website will access its built-in hybrid deep learning model to analyze the company's associated news and stock data for a certain amount of time. The deep learning model consists of a sentiment analysis model, which will be used to shift through news headlines and determine the connotation of the text, and a quantitative analysis model, which will shift through stock data to determine patterns within the numbers. Upon referring to its training and studying the current data, the model will make an educated prediction of where this company's stock will be headed. When it is finished analyzing the data, the model will return its findings to the website, where it will reveal the direction and explain to the user where the stock property is moving.

When we begin testing the software, we aim to monitor three stock properties over the course of three weeks to determine the code's accuracy. The website will undergo two rounds of testing: a preliminary round which will test the effectiveness of the website and a secondary round that addresses any issues within the first experiment. The experiment would be considered a success if the website is able to accurately determine the direction of these stock properties.

Timeline

Date	Task	Deliverables
12/5/23	Game Plan and Labor Distribution	N/A
12/12/2023 – 1/9/2023	Winter Break	- Sentiment Analysis Model - Deep Learning Model - Basic HTML Website
1/16/2024	Merging the Parts	Prototype of Stock Prediction Software

1/22/2024		
1/29/2024	Testing / Refinement	Stock Prediction Report Findings
2/5/2024		
2/12/2024		
2/19/2024	Troubleshooting	Troubleshooting and Update Documentation
2/26/2024		
3/4/2024	Second Round of Testing/Refinement	Stock Prediction Report Findings, Round 2
3/11/2024		
3/18/2024		
3/25/2024	Website Integration	HTML Website w/ Integrated Stock Prediction Software
4/1/2024		
4/8/2024	Presentation Preparation	- Completed Website - Presentation
4/15/2024	Symposium	Completed Presentation

Individual Parts and Labor

When building the software, there were four main parts associated with its creation alongside drawing up the literature. Ayomiro Aderin contributed to the artificial neural network program, drafted the necessary paperwork for the assignment, and built the host webpages for the stock analyzer. Cameron Graves conducted research on data scraping behaviors, served as one of the team's experts in stock market behavior, and assisted with constructing the main Python program that conjoined the ANN model with the sentiment analysis model and presented the trading options. Abdul Seidu constructed the sentiment analysis program, helped design the conditional function that determines trading action, and was another leading expert in stock market behavior.

Experiment

To ensure that the program works, the research team decided to watch five stock properties (AAPL, AMZN, META, TSLA, and ABNB) over the course of a business week. Our null hypothesis was that the website could not predict the trajectory of the stock market's prices and as such, would not be able to provide a useful course of action. Alternatively, the anticipation was that the stock market analyzer would accurately determine the pathway of a stock property

and provide a helpful suggestion. The findings were recorded within an Excel file, tracking the predicted price, accurate price, the predicted direction, and the actual direction. In addition, sum and percentage differences between the predicted and actual prices.

For the experiment, the user would submit the information of the ticker they are interested in examining. This would be sent to the training model method of the ANN code. To test, the established window for training the data ranged from January 1, 2020 and January 1, 2024. Once the model was created, the user would then enter the range of dates they would like to predict. The week in question was April 1, 2024 to April 5, 2024. Therefore, for each day inspected, the new range was from January 1, 2024 to April {date}, 2024. This would submit the information into the ANN model and the sentiment analysis model to gather their individual results. Upon receiving the input from these two models, the main program would utilize a conditional statement method to determine the best plan of action based upon the two machine learning algorithms.

Results

When analyzing the data provided by the experiment, the average sum difference between the predicted price and the actual price was $-\$0.75$, which equates to -0.43% . In addition, for three of the five properties tracked, it accurately determines the direction of the stock price. The sentiment analysis model accurately determined the mood of the financial news headlines to inform the user of the company's current reputation. When observing the suggested courses of action that the machine provided, it suggested that the user either buy or watch a stock property. This aversion to negative results could be attributed to the limited news headline window. Finally, after the week has passed, the predicted net price either overestimate the actual net price's loss or underestimated the net price's gain. Recognizing this could provide a level of

safety for investors who would like to know if they should pull their money or leave it within the market.

Recommendations

One way to improve the efficacy of the stock analyzer would be to include a stop-loss threshold for users. Given that the website's goal is to protect the financial assets of its users, it would benefit them greatly if they were able to submit their personal financial loss threshold to prevent them from losing what they've invested. In future developments, the website would include a database that permits for individual user logins so that an investor's profile would include their stop loss threshold when their trading. In this manner, the website could provide more informed counsel when users are trading using its information.

Another aspect of the website that could be improved is the source of financial news. Our website's source of news is FinViz, which has low data scraping protections. Unfortunately, the website only provides 100 headlines at a time, regardless of the breadth of the dates. While it was able to provide accurate, real-time headlines for us to examine and use, the limited scope skewed the focus of our sentiment analysis model. If further research were conducted, the team would include a condition that would allow for the news headlines to be filtered and associated with their dates in a way that a company's reputation could be included in the final analysis.

Finally, the ANN machine could utilize a more nuanced method of determining the direction of the stock market price. When creating the method, we've decided to utilize the percentage difference between closing prices to signify the direction of a stock property. This general approach may have proven instrumental in providing some of the more erroneous suggestions. Should further research be conducted, an alternative way of tracking stock

movement would be suggested. One of these methods include studying moving averages, a topic brought up when discussing the creation of the artificial neural network.

Conclusion

Through creating the stock analyzer, it's become clear that it could stand to benefit users with limited experience with the stock market and those who don't have the time to analyze stock patterns and charts to make smarter financial decisions. Machine learning algorithms such as ANN and sentiment analysis have proven beneficial in studying the patterns of the stock market, though with more developments, this prototype could serve as a reliable financial advisor for individuals interested in participating in stock market trading.

Predictive analysis has become a useful way for technology to understand how humans interact with the world. Through the examination of the stock market, the project anticipates a greater understanding of how well deep learning algorithms can predict human behavior when given proper context. By creating a hybrid model using sentiment analysis and linear regression algorithms, the research project aims to further studies based in the realm of machine learning. Ultimately, these algorithms help computers develop the same connections humans naturally make when studying data. Should this experiment end in success, machine learning models will be that much closer to understanding more complex patterns of data that humans are still learning to decipher.

Works Cited

- [1] Singh, Gurjeet. "Machine learning models in stock market prediction." arXiv preprint arXiv:2202.09359 (2022).

- [2] Soni, Payal, Yogya Tewari, and Deepa Krishnan. "Machine Learning approaches in stock price prediction: A systematic review." Journal of Physics: Conference Series. Vol. 2161. No. 1. IOP Publishing, 2022.

- [3] Krishna, Somanchi Hari, et al. "The Use of Deep Learning for Predictive Analytics in Financial Management." 2023 3rd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE). IEEE, 2023.

- [4] Sigo, Marxia Oli, et al. "Application of ensemble machine learning in the predictive data analytics of indian stock market." Webology 16.2 (2020): 2019.

Appendix A – Artificial Neural Network

```
import numpy as np
import pandas_datareader as web
import joblib
from joblib import dump, load
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential, load_model
import matplotlib.pyplot as plt
from keras.layers import Dense, LSTM
import yfinance as yf
import math
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

plt.style.use('fivethirtyeight')

class LSTMModel:
    def __init__(self, name=None):
        self.model = None
        self.name = name
        self.scaler = None # Add scaler attribute

    def load_model(self, model_file_path):
        self.model = load_model(model_file_path)

    def load_scaler(self, scaler_file_path):
        self.scaler = joblib.load(scaler_file_path)
```

```
def train_model(self, ticker, start_date, end_date):

    try:

        # Fetch historical data including opening, high, low, close, and volume
        df = yf.download(ticker, start=start_date, end=end_date)

    except ValueError:

        print(f'Error: Failed to download data for {ticker}. Please check the ticker symbol and try again.')

        return

    # Preprocess the data to include all relevant features
    dataset = df[['Open', 'High', 'Low', 'Volume', 'Close']].values
    print(dataset)

    # Initialize scaler
    self.scaler = MinMaxScaler(feature_range=(0,1))

    # Normalize the data
    scaled_data = self.scaler.fit_transform(dataset)

    print("+++++")
    print(scaled_data)
    print("+++++")
    X = scaled_data[:,0:4]
    print(X)
    print("+++++")
    Y = scaled_data[:,4]
    print(Y)
    x_train, x_test, y_train, y_test = train_test_split(X, Y)
    x_train_size = len(x_train)
    print(x_train)
```

```
print(y_train)
AA = np.size(x_train)
print("+++++")

x_train = x_train.reshape(x_train_size, 1, 4)

self.model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(1, 4)),
    LSTM(50, return_sequences=False),
    Dense(25, activation='relu'),
    Dense(1, activation='linear')
], name='MyLSTMModel') # Set the name attribute here

# Compile and train the model
self.model.compile(optimizer='adam', loss='mean_squared_error')
self.model.fit(x_train, y_train, batch_size=32, epochs=50)

# Save the trained model
self.model.save(f'media/lstm_model.h5')

# Save the scaler
scaler_file_path = f'media/scaler.pkl'
joblib.dump(self.scaler, scaler_file_path)

def predict_direction(self, pred_price, last_price):
    price_difference = pred_price - last_price
    percentage_difference = (price_difference / last_price) * 100

    if abs(percentage_difference) <= 0.5:
        direction = "Stall"

    elif 0.5 < percentage_difference <= 2:
```

```
        direction = "Rise" if percentage_difference > 0 else "Fall"
    elif 2 < percentage_difference <= 5:
        direction = "Sharp Rise" if percentage_difference > 0 else "Sharp Fall"
    elif percentage_difference > 5:
        direction = "Significant Rise" if percentage_difference > 0 else "Significant Fall"
    else:
        direction = "Neutral" # Default direction

    return direction

def predict_price(self, ticker, start_date, end_date):
    if self.model is None:
        print("Error: Model not trained yet.")
        return None, None # Return None for both direction and price

    try:
        # Fetch historical data
        df = yf.download(ticker, start=start_date, end=end_date)

        # Preprocess the data
        dataset = df[['Open', 'High', 'Low', 'Volume', 'Close']].values
        scaled_data = self.scaler.transform(dataset) # Use the same scaler instance used for training

        A = scaled_data[:, :4]
        print(A)
        print("+++++")
        B = scaled_data[:, 4]
        print(B)
        x_train, x_test, y_train, y_test = train_test_split(A, B)
        x_test_size = len(x_test)
```

```
print(x_test)
print(y_test)
AA = np.size(x_test)
print("+++++++")

A = A.reshape(A.shape[0], 1, A.shape[1])

# Make predictions
predictions = self.model.predict(A)
print("Shape of predictions:", predictions.shape) # Print shape of predictions
predictions = self.scaler.inverse_transform(np.concatenate((A.reshape(A.shape[0], A.shape[2]),
predictions), axis=1))[:, -1] # Use the same scaler instance

# Get the last actual price
last_price = df.iloc[-1]['Close']

# Predict direction
direction = self.predict_direction(predictions[-1], last_price)

df = df.loc[start_date:end_date] # Limiting the data to the specified time range

# Save plot
plt.figure(figsize=(16,8))
plt.title(f'{ticker} Model')
plt.xlabel('Date', fontsize=18)
plt.ylabel('Close Price USD ($)', fontsize=18)
plt.plot(df.index, df['Close'], label='Actual') # Plot actual closing prices
plt.plot(df.index, predictions, label='Predicted') # Plot predicted closing prices
plt.legend(loc='lower right')
plot_file_path = f'media/model_plot.png' # Include ticker symbol in the plot file name
```

```

plt.savefig(plot_file_path)

plt.close()

# Save the trained model
model_file_path = f'media/lstm_model.h5' # Include ticker symbol in the model file name
self.model.save(model_file_path)

predicted_price_formatted = "${:.2f}".format(round(predictions[-1], 2))
return direction, predicted_price_formatted
except ValueError:
    print(f'Error: Failed to download data for {ticker}. Please check the ticker symbol and try again.')
    return None, None

if __name__ == "__main__":
    model = LSTMMModel()
    ticker_to_train = input("Enter a ticker to train the model: ")
    start_date = input("Enter the start date for training (YYYY-MM-DD): ")
    end_date = input("Enter the end date for training (YYYY-MM-DD): ")
    model.train_model(ticker_to_train, start_date, end_date)

    tickers_to_predict = input("Enter tickers separated by commas to predict their directions: ").split(',')
    predict_start_date = input("Enter the start date for prediction (YYYY-MM-DD): ")
    predict_end_date = input("Enter the end date for prediction (YYYY-MM-DD): ")
    for ticker in tickers_to_predict:
        direction, predicted_price = model.predict_price(ticker.strip(), predict_start_date, predict_end_date)
        if direction is not None:
            print(f'Predicted direction for {ticker}: {direction}')
            print(f'Predicted price for {ticker}: {predicted_price}')

```

Appendix B – Sentiment Analysis

```
from urllib.request import urlopen, Request
from bs4 import BeautifulSoup
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import pandas as pd
import datetime

class SentimentAnalysisModel:
    def __init__(self):
        nltk.download('vader_lexicon')
        self.vader = SentimentIntensityAnalyzer()
        self.finviz_url = 'https://finviz.com/quote.ashx?t='
        self.today_date = datetime.datetime.today().strftime('%b-%d-%y')
        self.news_tables = {}

    def fetch_news_data(self, tickers):
        for ticker in tickers:
            url = self.finviz_url + ticker
            req = Request(url=url, headers={'user-agent': 'my-app'})
            response = urlopen(req)
            html = BeautifulSoup(response, features="lxml")
            news_table = html.find(id='news-table')
            self.news_tables[ticker] = news_table

    def parse_news_data(self):
        parsed_data = []
        last_date = None
```

```
for ticker, news_table in self.news_tables.items():
    if news_table:
        for row in news_table.findAll('tr'):
            title_cell = row.a
            if title_cell:
                title = title_cell.text
                cells = row.findAll('td')
                if len(cells) > 1:
                    date_time_str = cells[0].text
                    date_time_list = date_time_str.split()

                    if len(date_time_list) >= 2:
                        date = date_time_list[0]
                        time = date_time_list[1]
                    else:
                        date = date_time_list[0]
                        time = 'N/A'

                    if date == 'Today':
                        date = self.today_date

                    if last_date is None or date != last_date:
                        last_date = date

                # Calculate sentiment score
                sentiment_score = self.vader.polarity_scores(title)['compound']

                # Categorize sentiment
                if sentiment_score >= 0.05:
                    sentiment_category = 'Positive'
```



```
        elif sentiment_score <= -0.05:
            sentiment_category = 'Negative'
        else:
            sentiment_category = 'Neutral'

    parsed_data.append([ticker, last_date, time, title, sentiment_score, sentiment_category])

df = pd.DataFrame(parsed_data, columns=['ticker', 'date', 'time', 'title', 'sentiment_score',
'sentiment_category'])
df['date'] = pd.to_datetime(df['date']) # Convert date column to datetime type

return df

# Inside analyze_sentiment method of SentimentAnalysisModel class
def analyze_sentiment(self, ticker):
    self.fetch_news_data([ticker])
    df = self.parse_news_data()

    df['date'] = pd.to_datetime(df['date'], format='%b-%d-%y')

    # Use .loc to avoid SettingWithCopyWarning
    df.loc[:, 'sentiment_score'] = df['title'].apply(lambda x: self.vader.polarity_scores(x)['compound'])

    def categorize_sentiment(compound_score):
        if compound_score >= 0.05:
            return 'positive'
        elif compound_score <= -0.05:
            return 'negative'
        else:
```

```
return 'neutral'
```

```
df.loc[:, 'sentiment_category'] = df['sentiment_score'].apply(categorize_sentiment)
```

```
sentiment_counts = df['sentiment_category'].value_counts()
```

```
total_articles = len(df)
```

```
sentiment_counts_percentages = {
    'Positive': (sentiment_counts.get('positive', 0) / total_articles) * 100,
    'Negative': (sentiment_counts.get('negative', 0) / total_articles) * 100,
    'Neutral': (sentiment_counts.get('neutral', 0) / total_articles) * 100,
    'Other': ((total_articles - sum(sentiment_counts)) / total_articles) * 100
}
```

```
return sentiment_counts_percentages
```

```
def main(self):
```

```
    ticker = input("Enter a ticker symbol: ")
```

```
    self.fetch_news_data([ticker])
```

```
    df = self.parse_news_data()
```

```
    df['date'] = df['date'].dt.strftime('%Y-%m-%d')
```

```
    df.to_csv('media/news_data.csv', index=False) # Save all data to a CSV file in 'media' directory
```

```
    writer = pd.ExcelWriter('media/news_data_2.xlsx', engine='xlsxwriter')
```

```
df.to_excel(writer, index=False, sheet_name='Sheet1')

workbook = writer.book
worksheet = writer.sheets['Sheet1']

date_format = workbook.add_format({'num_format': 'yyyy-mm-dd'})
worksheet.set_column('A:A', None, date_format)
writer._save()

sentiment_breakdown = self.analyze_sentiment(ticker)
if sentiment_breakdown:
    print("Sentiment Analysis Breakdown:")
    for sentiment, percentage in sentiment_breakdown.items():
        print(f'{sentiment.capitalize()}: {percentage:.2f}%')
    most_popular_sentiment = max(sentiment_breakdown, key=sentiment_breakdown.get)
    print("Most Popular Sentiment:", most_popular_sentiment.capitalize())

if __name__ == "__main__":
    SentimentAnalysisModel().main()

I.      Main Program Code - Python
```

Appendix C – Main Program

```
import sent_anal_3
import ann_lstm_multitrain

def train_model(model, train_ticker, train_start_date, train_end_date):
    model.train_model(train_ticker, train_start_date, train_end_date)
    return model

def analyze_sentiment(ticker):
    sentiment_model = sent_anal_3.SentimentAnalysisModel()
    return sentiment_model.analyze_sentiment(ticker)

def predict_stock_price(model, ticker, start_date, end_date):
    return model.predict_price(ticker, start_date, end_date)

def determine_action(sentiment_breakdown, lstm_result):
    if lstm_result is not None:
        if "Rise" in lstm_result[0]:
            stock_movement = "up"
        elif "Fall" in lstm_result[0]:
            stock_movement = "down"
        else:
            stock_movement = "stagnant"

    most_popular_sentiment = max(sentiment_breakdown, key=sentiment_breakdown.get)
    news = most_popular_sentiment

    if news == "positive":
```

```
    if stock_movement == "up":

        return f'It's strongly recommended that you buy this stock. The news is {news} and the stock
price is going {stock_movement}.'"

    elif stock_movement == "down":

        return f'It is not recommend that you buy this stock. While the news is {news}, the stock price
is going {stock_movement}.'"

    else:

        return f'It's recommended that you buy this stock. While the stock price is {stock_movement},
the news is {news}.'"

    elif news == "negative":

        if stock_movement == "up":

            return f'It's recommended to cautiously buy this stock. While the stock price is
{stock_movement}, the news is {news} '"

        elif stock_movement == "down":

            return f'It's strongly recommended that you sell this stock. The news is {news} and the stock
price is going {stock_movement}.'"

        else:

            return f'It's recommended that you sell this stock. While the news is {news}, the stock price is
{stock_movement}.'"

    else: # Neutral news

        if stock_movement == "up":

            return f'It's recommended that you buy this stock. While the news is {news}, the stock price is
going {stock_movement}.'"

        elif stock_movement == "down":

            return f'It's recommended that you sell this stock. While the news is {news}, the stock price is
going {stock_movement}.'"

        else:

            return f'It's recommended that you watch this stock carefully. The news is {news} and the stock
price is {stock_movement}.'"

    else:

        return "Prediction failed. Please check the inputs and try again."

def main():
```

```

last_trained_ticker = None # Variable to store the last trained ticker symbol
model = None # Initialize model outside the loop

while True:
    train_ticker = input("Enter a ticker to train the model: ")

    # Check if the ticker for training is different from the last trained ticker
    if train_ticker != last_trained_ticker or model is None:
        train_start_date = input("Enter the start date for training (YYYY-MM-DD): ")
        train_end_date = input("Enter the end date for training (YYYY-MM-DD): ")
        model = ann_lstm_multitrain.LSTMMModel()
        model = train_model(model, train_ticker, train_start_date, train_end_date)
        last_trained_ticker = train_ticker # Update last trained ticker
        print("Model trained successfully.")
    else:
        print("Using existing model. No need to retrain.")

    ticker = input("Enter a ticker symbol: ")
    start_date = input("Enter the start date (YYYY-MM-DD): ")
    end_date = input("Enter the end date (YYYY-MM-DD): ")

    sentiment_breakdown = analyze_sentiment(ticker) # Corrected line
    lstm_result = predict_stock_price(model, ticker, start_date, end_date)

    if lstm_result is not None: # Check if lstm_result is not None
        print("Sentiment Analysis Breakdown:")
        for sentiment, percentage in sentiment_breakdown.items():
            print(f"{sentiment}: {percentage:.2f}%")
        most_popular_sentiment = max(sentiment_breakdown, key=sentiment_breakdown.get)
        print("Most Popular Sentiment:", most_popular_sentiment)

```

```
print("LSTM Model Prediction:", lstm_result)

action = determine_action(sentiment_breakdown, lstm_result)
print("Action to take:", action)

exit_choice = input("Do you want to continue (Yes/No)? ").lower()
if exit_choice == "no":
    break

if __name__ == '__main__':
    main()
```